

Mastering Core Data Structures & Asymptotic Complexity

Department of Computer Science & Engineering | Comprehensive Reference Guide

1. Asymptotic Complexity (Big-O Notation)

Big-O notation describes the upper bound of an algorithm's execution time or space requirements as input size n grows asymptotically. The fundamental hierarchy from fastest to slowest:

Notation	Classification	Typical Examples	Scalability (10 ⁶ items)
$O(1)$	Constant	Hash Map lookup, Array index access	Instantaneous (~1 op)
$O(\log n)$	Logarithmic	Binary Search on sorted array	~20 operations
$O(n)$	Linear	Linear search, array traversal	1,000,000 operations
$O(n \log n)$	Linearithmic	MergeSort, QuickSort (avg), HeapSort	~20,000,000 operations
$O(n^2)$	Quadratic	Bubble Sort, Nested pair comparisons	1,000,000,000,000 ops (slow)
$O(2^n)$	Exponential	Recursive Fibonacci, Subset generation	Intractable for large n

2. Binary Search Trees (BST) & Self-Balancing Trees

A binary search tree satisfies the BST property: for every node X , all keys in its left subtree are less than $X.key$, and all keys in its right subtree are greater than $X.key$. Average search is $O(\log n)$, but pathological degenerate cases can degrade to $O(n)$ without balancing (e.g. AVL, Red-Black Trees).

Key Operations & Complexity:

- Search: $O(\log n)$ balanced, $O(n)$ skewed
- Insert: $O(\log n)$ balanced
- Delete: $O(\log n)$ balanced
- In-Order Traversal yields sorted sequence in $O(n)$ time.